

Testing some aspects of Object-Oriented Ontology using an Open Source Software Tool.

Robin Baines, July 2026

1 Abstract

Object-Oriented Ontology (OOO) is a recent philosophical movement that rejects human-centric thinking. Developed by thinkers like Graham Harman, Ian Bogost, and Timothy Morton, OOO emerged as a branch of Speculative Realism.

The WOOOF program is an open source program which converts a scenario into an object model.

I have used the WOOOF program to test a limited set of OOO ideas on three scenarios which describe interactions between objects without an explicit observer. This has resulted in a uniform approach which confirms that OOO is useful for analysis.

Compound objects come into existence and disintegrate during interactions. Avoiding internal hierarchy in these objects avoids complications and supports the OOO idea of a flat ontology. However OOO seems to suggest that an object could include another object and retain its identity. When considering, for example, coalescence of water drops this seems to imply that there is an arbitrary boundary, on the relative size of drops.

In these tests time is divided it into periods of gradual change punctuated by events when changes occur to objects instantaneously. The events are used to implement the OOO idea of vicarious causation.

A second type of scenario introduces two observers, a human and a dog both of which perceive and influence each other. Parallel sub-scenarios are used to represent a delayed response of the human to an action taken by the dog and vice versa. A simple object model is introduced to organise pre-existing information in the observers.

The four scenarios support aspects of OOO but also extend it by including instantaneous events and by introducing several objects types in observers.

Table of Contents

1 Abstract.....	1
2 Introduction.....	3
3 How the WOOOF language has been used to express OOO ideas.....	4
3.1 The INIT object.....	4
3.2 Composition.....	4
3.3 Inheritance.....	6
3.4 Visibility of Properties.....	7
3.5 Time and Events.....	7
3.6 Object Lifecycle.....	9
3.7 Independent Scenarios.....	9
3.8 Level of Detail.....	10
3.9 Compound Objects.....	11
3.10 Bringing it together: drops scenario as an Example.....	13
3.11 Instantiation.....	14
4 A Scenario with Observers.....	15
4.1 The dogs Scenario.....	15
4.2 Object hierarchy inside an Observer.....	16
4.3 Observers and OOO.....	17
5 Do the scenarios support OOO?.....	17
5.1 Level of Detail and Withdrawal.....	17
5.2 Timelines and Events. Two types of object?.....	17
5.3 A Generic Approach to Inanimate Interactions.....	17
5.4 Compound Objects.....	17
6 If OOO does describe reality or, at the very least, is a useful analysis tool; what questions arise from its success?.....	18
6.1 Why do Objects Work Outside and Inside an Observer?.....	18
6.2 An 'interface' Object which Observers use when Interpreting Reality.....	18
7 Conclusion.....	18

Table of Figures

Figure 1: drops.txt.....	3
Figure 2: golf.txt.....	3
Figure 3: PoolAndStone.txt.....	4
Figure 4: The Real Object called RO_bigdrop is included in the SO called bigdrop.....	5
Figure 5: The club_ball compound object.....	5
Figure 6: Presentation of Inheritance.....	6
Figure 7: My_Nida is a dog. dog is an animal.....	6
Figure 8: The drops scenario showing the INIT object, the collision timeline and the COLLISION Event.....	8
Figure 9: the club_ball object is Greyed out, the separate club and ball objects survived the scenario.....	9
Figure 10: PoolAndStone scenario showing Independent sub-scenarios: Air stops moving in pl1 and Stone falling in water in pl2.....	10
Figure 11: The Stone touches the Water and the stone_air_water Compound object is created.....	12
Figure 12: WOOOF output of drops.txt.....	14
Figure 13: SEE_SOMETHING black moving.....	15
Figure 14: My_Nida contains Today_Nida and Today_Nida is a type of My_Nida.....	16

2 Introduction.

Object-Oriented Ontology (OOO) is a recent philosophical movement that rejects human-centric thinking. OOO asserts that all objects exist independently of human perception and share an equal ontological status. An object is fundamentally "withdrawn" and never fully knowable or reducible. Developed by thinkers like Graham Harman, Ian Bogost, and Timothy Morton, OOO emerged as a branch of Speculative Realism. See [1] Object-oriented Ontology: A New Theory of Everything by Graham Harman. And [2] Waves and Stones: On the Ultimate Nature of Reality by Graham Harman.

Several Object Oriented Ontology (OOO) ideas are:

- Real Objects (ROs) with Real Qualities (RQs) and Sensual Objects (SOs) with Sensual Qualities (SQs) exist.
- Objects form a flat ontology implying, among other things, that interaction between inanimate objects without an observer is valid.
- Vicarious causation: object interaction is facilitated by a third object.
- Undermining; a real object cannot be reduced downward to only its parts.
- Overmining; a real object has a surplus beneath the sum total of effects.
- Under- and Over- mining require that an object is more than its pieces and less than its effects.

The WOOOF program is an open source program which converts a scenario into an object model. It is available from <https://github.com/RobinBaines/OOO3/releases/tag/v1.0.4>. The handbook is in the Readme.md file on the site.

The WOOOF language uses some Object Oriented Programming (OOP) ideas; Inheritance, Properties and Composition. WOOOF allows experiments with different approaches to modelling objects and object interaction. I have used the WOOOF program to test several OOO ideas by modelling interactions between inanimate objects in three scenarios:

1. drops.txt Two drops of water coalesce. The resulting drop splits in a gust of wind.

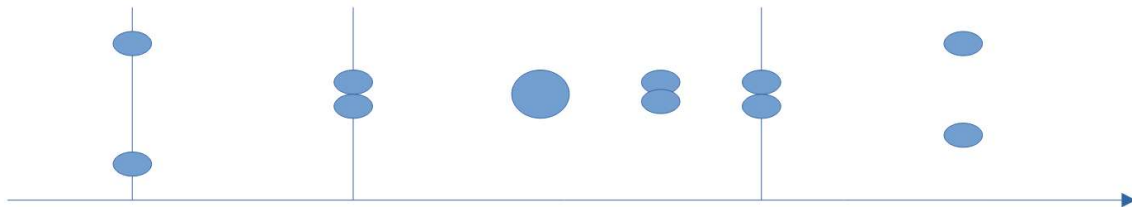


Figure 1: drops.txt

2. golf.txt A golf club hits a golf ball and the club and ball form a new object. After a period of intense energy transfer the club and ball separate.

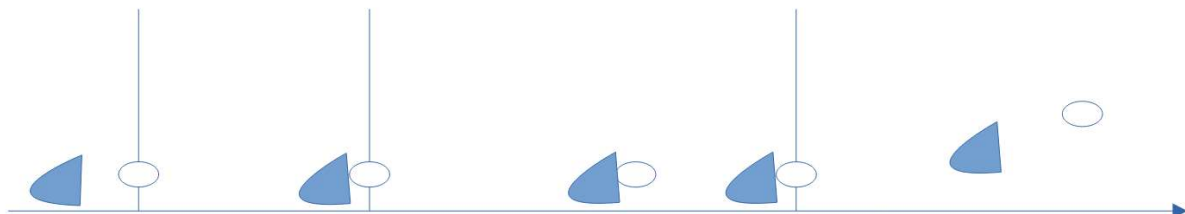


Figure 2: golf.txt

3. PoolAndStone.txt A stone falls towards the surface of a pool of water. The stone enters the water and comes to rest at the bottom of the pool. The air which was disturbed by the passage of the stone comes to rest. The water which was disturbed by the passage of the stone comes to rest.

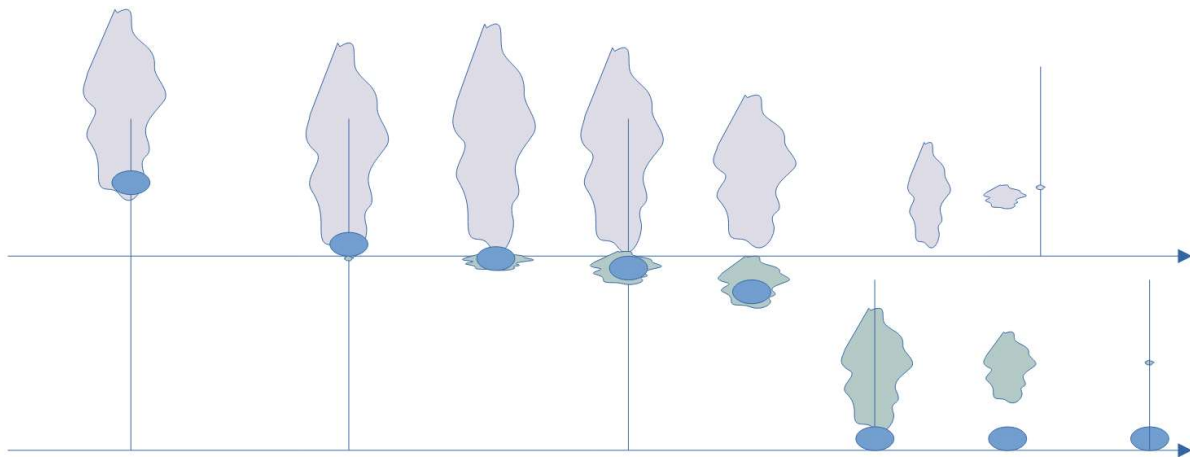


Figure 3: PoolAndStone.txt

A forth scenario, dogs.txt, introduces two observers, a human called 'Me' and a dog called Nida.
 A dog is sitting then running and then barking at an area where dogs can run freely.
 The human recognises the running dog and later hears it barking.
 The human commands the dog to sit.
 The dog recognises the command, stops barking, stops running and sits.

The following scenarios are part of the WOOOF release on github.

- ACW.txt describes the American Civil War and is an early script which was inspired by Graham Harman's analysis and ideas on object development using symbiosis.
- PoolAndStoneObject.txt is a version the PoolAndStone.txt script using an alternative approach to compound objects.

The first chapter describes how the WOOOF language has been used to express OOO ideas in the three inanimate scenarios. The second chapter shows the effect of adding observers. The following chapter addresses whether the scenarios support OOO. As this chapter is positive about OOO it is followed by a discussion headed "If OOO describes reality or, at the very least, is a useful analysis tool; what questions arise from its success?"

3 How the WOOOF language has been used to express OOO ideas.

The scenarios can be expressed using the WOOOF language in several ways and finding scenarios which had a common approach and which apply OOO ideas consistently involved several revisions. The WOOOF language is used to define a scenario and the WOOOF program to display it. The following summarises the findings.

3.1 The INIT object.

The INIT object is the first object in a script and is used to introduce and initialise other objects. This convention avoids having to describe how each object has come into existence. For example it would be tedious to describe how the two drops in drops.txt came into existence via a series of events involving nucleation, condensation and collisions.

The INIT object is useful for practical reasons related to scripting. It has similarities with the definition of initial conditions in engineering models.

3.2 Composition

In WOOOF composition uses the 'includes' and 'include_right' commands. For example 'Me includes My_Nida'. In OOP composition can also be expressed as an Object 'has another' object; for example, a car has a steering wheel. The include commands have been used to implement the OOO idea that an object can have parts which are also objects. More surprising is that it has been used to express the relationship between an SO and an RO, see Figure 4.

```

bigdrop(gust_of_wind):SO
bigdrop(COLLISION) => INHERIT SO True
bigdrop(COLLISION) => mass 2gm true
ENDED BY: INPUT2

RO_bigdrop(gust_of_wind):RO
RO_bigdrop(COLLISION) => INHERIT RO True
ENDED BY: INPUT2

```

Figure 4: The Real Object called RO_bigdrop is included in the SO called bigdrop.

The display in WOOOF shows how the SO bigdrop can be thought of as a 'wrapper' object which encapsulates and protects the RO called RO_bigdrop from external access. This is a reasonable match with the OOO concept of the withdrawal of real objects.

An object which is composed of two or more other objects is called a Compound object, see the following figure of the club_ball Compound object which occurs in the golf.txt scenario.

```

club_ball(COLLISION):SO
INHERIT SO True
=> OUTPUT1.mass 800gm true
=> OUTPUT1.x = 0
=> OUTPUT1.y = 0
=> OUTPUT1.z = 0
=> OUTPUT1.v = 0
ENDED BY: club_ball_contact

RO_club_ball(COLLISION)
ENDED BY: club_ball_contact

club(COLLISION):SO
club(the_swing) => INHERIT SO True
club(the_swing) => mass 800gm true
club(the_swing) => x = -100
club(the_swing) => y = 0
club(the_swing) => z = 0
club(the_swing) => v = 100
ENDED BY: club_ball_contact

RO_club(COLLISION):RO
RO_club(INIT) => INHERIT RO True
ENDED BY: club_ball_contact

ball(COLLISION):SO
ball(the_swing) => INHERIT SO True
ball(the_swing) => mass 200gm true
ball(the_swing) => x = 0
ball(the_swing) => y = 0
ball(the_swing) => z = 0
ball(the_swing) => v = 0
ENDED BY: club_ball_contact

RO_ball(COLLISION):RO
RO_ball(INIT) => INHERIT RO True
ENDED BY: club_ball_contact

```

Figure 5: The club_ball compound object.

3.3 Inheritance.

Inheritance is an OOP term: Child classes may inherit (or be derived) from a parent class. The children are usually 'a type of' the parent; or the child 'is a' parent. WOOOF supports inheritance using the 'type' or 'type of' command.



Figure 6: Presentation of Inheritance

The figure shows how COLLISION is a type of SO_EVENT.

3.4 Properties

The scripts are representations of how objects could relate to each other and to keep the scripts short, properties have been kept to a minimum. Scripts of inanimate interactions include coordinate and velocity properties chosen to ensure that the drops collide. In retrospect this looks like an artifact of scripting and real interactions would not need this sort of decoration.

3.5 Time and Events.

Time increases from top to bottom in the text file in which a scenario is defined and from left to right in the WOOOF display.

Time has two implementations:

- A 'Timeline' is a Sensual Object with a finite duration. It is derived from an object called RO_timeline, which is a single RO representing the time continuum.
- A discrete Event is a Real Object with no duration and is unique. ("un momento dado"; Johan Cruyff). An Event can involve no more than two objects because it is instantaneous. Potentially there could be a none countable, infinite number of Events. In practice an Event is instantiated when an object ends or an object becomes part of a new object or if a property of an object changes. By convention Events names are in capital letters.

An Event may have an input and/or an output object called INPUTx and OUTPUTx, by convention; see the drops scenario in the following figure. The Real Object called RO_COLLISION is contained within the SO called COLLISION.

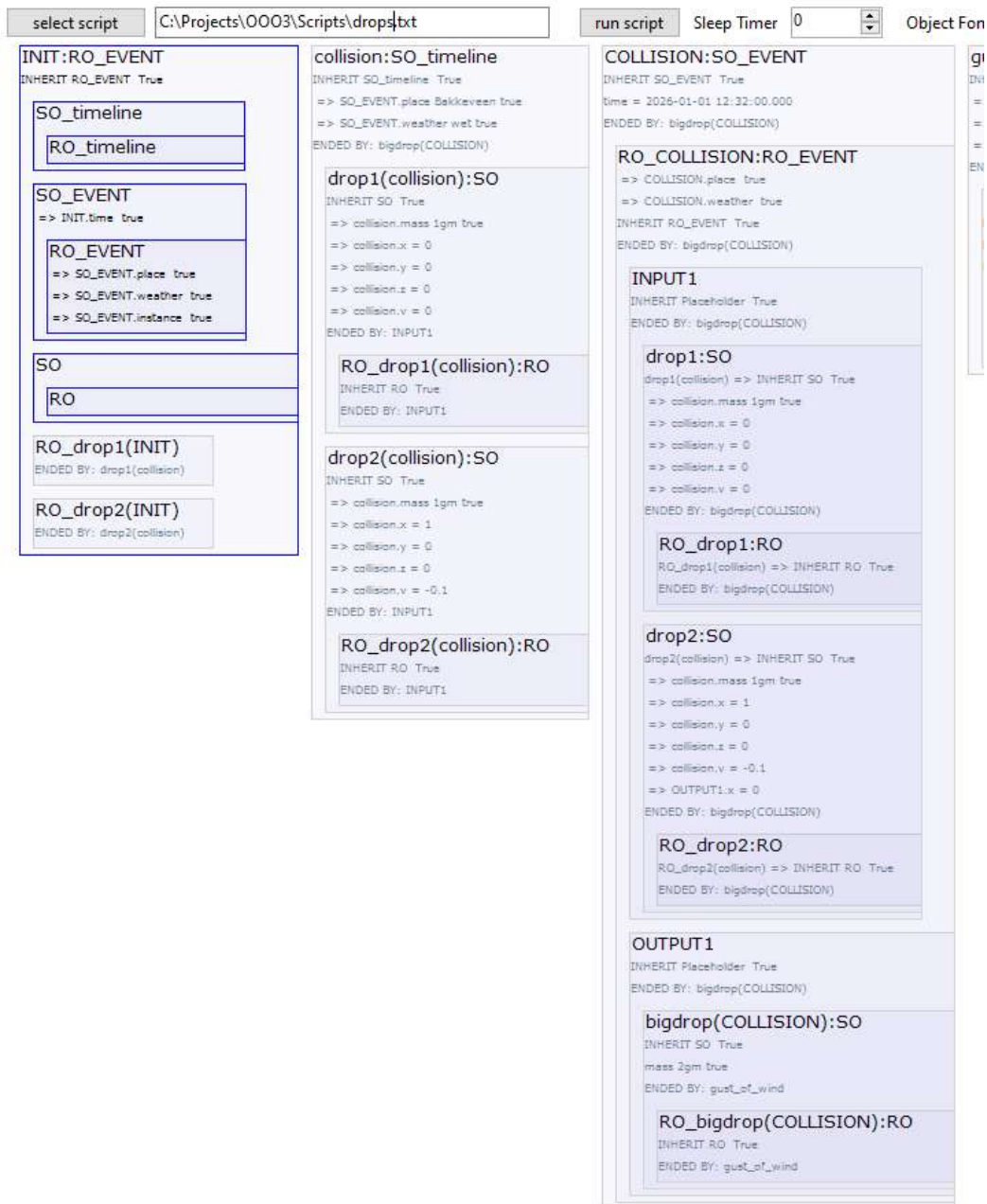


Figure 7: The drops scenario showing the INIT object, the collision timeline and the COLLISION Event.

Events occur at the beginning and end of a Timeline and it is the time stamps of the SO Events which mark the start and the end of the enclosed Timeline.

This approach to time and the introduction of the Event was a consequence of avoiding simultaneous existence of objects which enter an Event and the modified objects leaving the Event, for example, in the drops scenario 2 drops enter THE_COLLISION Event and one coalesced drop, called bigdrop, leaves it. In the scenarios an Event is the vicarious RO which facilitates interaction between objects. Difficulties in defining exactly when an Event occurs looks like an example of the OOO concept of withdrawal.

In some other scenarios very detailed Events may be justified. For example Brownian motion and a drops script with atomisation, instead of coalescence, would be scripted using many Events separated by short Timelines, the assumption being that no two events occur in the same instant. This assumption looks safe enough and was perhaps one of the insights which resulted in the "random walk" and the correct analysis of Brownian Motion by Albert Einstein.

The word event is most often used to describe a visit to the theatre or a football match. In WOOF the meaning is closer to the idea of an event in a C# program, for example, a mouse click or a file has just been read. An alternative name is Operator. In quantum mechanics, operators represent mathematical instructions like measuring position or momentum and act instantaneously.

3.6 Object Lifecycle.

If an Object, for example My_Nida in dogs.txt, has been included in a TimeLine and is then included in the next Event, the My_Nida SO is reproduced by the WOOF program and the version in the Timeline SO is greyed out. As the object proceeds through the Timelines and Events its properties change but the copies in preceding Timelines and Events do not. We can think of the greyed out objects as 'snapshots' made at the moment the containing Timeline or Event ends. If an object does not end within the scenario it will not be greyed out in the last TimeLine; it has survived the scenario!

The following figure shows the Compound object club_ball greyed out in the SPLIT Event. After the split, the separated club and ball objects still exist and are therefore not greyed out.

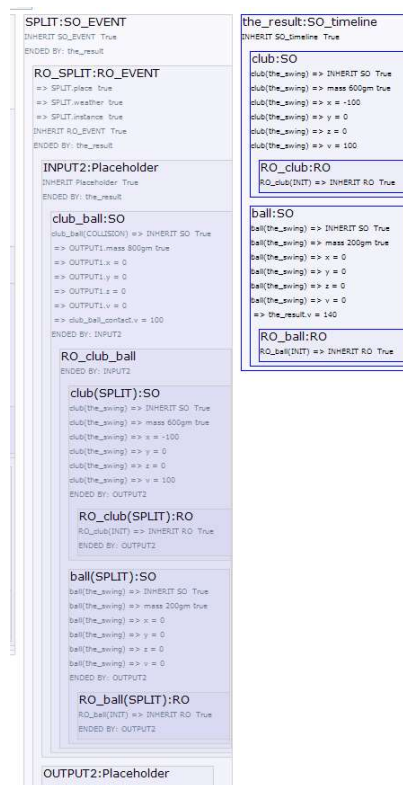


Figure 8: the club_ball object is Greyed out, the separate club and ball objects survived the scenario.

3.7 Independent Scenarios.

Several independent scenarios can occur within the main scenario. This idea is used in the PoolAndStone script: At a certain moment the stone is fully submerged and it continues to fall through the water. Independently, the air, which was disturbed earlier by the passage of the stone above the water, slowly comes to rest. The stone sinking in water and the air coming to rest are two independent scenarios and are displayed in two independent 'placeholder' objects. See pl2 and pl1 in the following figure.

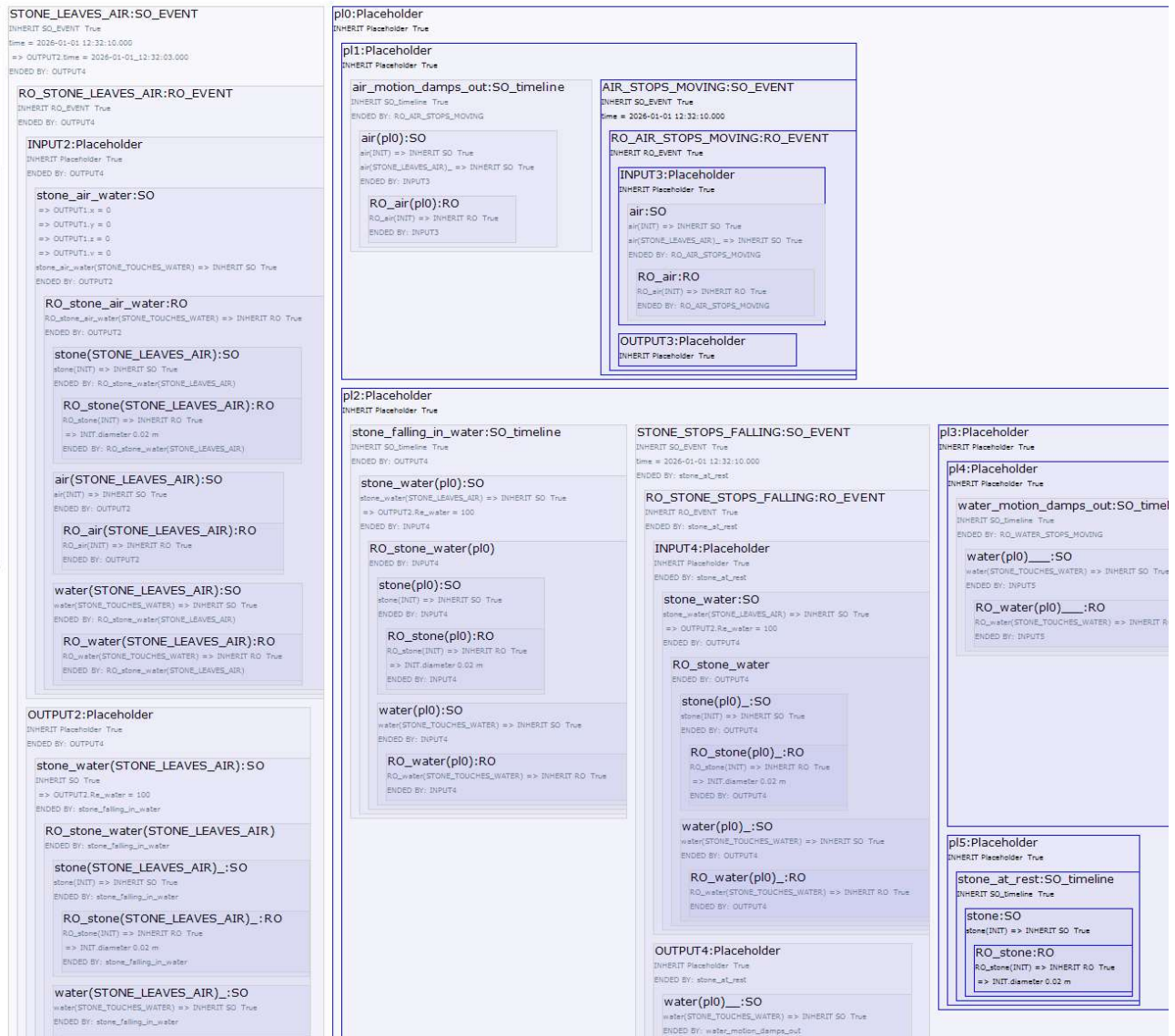


Figure 9: PoolAndStone scenario showing Independent sub-scenarios: Air stops moving in p1 and Stone falling in water in p2.

3.8 Level of Detail.

Timelines and Events are related to paradoxes associated with Zeno of Elea; if we are tempted to dive into ever more detail in a scenario then we could end up with so many events that no progress is made. The OOO idea of withdrawal is also a warning that this is not a good approach and what we may be attempting to achieve will remain just out of reach.

Here are some examples of how Events have been used in the scripts:

- In the drops script 2 drops of water collide and form a new drop in the Event called THE_COLLISION.
- In the golf script the club and ball make contact in an Event which is also called THE_COLLISION and form a new compound object called club_ball.
- In the PoolAndStone script the Event, called STONE_TOUCHES_WATER, is the moment the stone makes contact with the pool of water.

Defining the moment an Event takes place has some similarity with finding a maximum or minimum on a smooth curve without using differentiation. We know there is a maximum and can choose a position on a curve before the maximum and another after. Then by stepping from both directions the length of the curve on which the maximum must be gets shorter and shorter. In a similar way, in the drops script, we know there must be an instant when two drops coalesce to form a single drop, but have trouble finding it. and defining the exact timestamp as it is likely to have an infinite number of decimals!

What then should the level of detail be in a scenario?

The division of time into Timelines and instantaneous Events, has the potential for introducing ever more detail. In a similar way other types of object can be split into ever more detail, potentially to atoms and further. When considering inanimate scripts it looks difficult to fix this granularity without slipping in an observer to define the intention of the analysis. An observer could, for example, conclude that there is no point in creating Events for the arrival of every photon on the surface of a pollen particle when looking at Brownian motion. Events seem also to be associated with a change in the rate of energy transfer and the same conclusion is reached by comparing the rate of energy transfer caused by a photon and the collision of a molecule of water with the pollen.

Changes in the structure of objects and step changes in the rate of energy transfer seem to be pointers to a consistent level of granularity and emphasises that the OOO concept of withdrawal applies to time objects as well as physical objects. Withdrawal can be seen as an incentive to finding a practical point at which to stop digging deeper in scenarios.

3.9 Compound Objects

The last decision in scripting scenarios was how to treat Compound objects. In scenarios, Compound objects are formed in Events and because an Event is instantaneous there can never be more than two objects in the Input or Output objects of an Event. Compound objects are therefore always created from two objects, which may or may not be Compound objects themselves.

Should a new Compound object be a new type of object or should one of the 2 objects, let's call it the dominant object, include the other one because the essential nature of the dominant object has not changed?

In the drops scenario it looks convincing that the Compound drop formed when 2 smaller drops of similar size, coalesce is a new type of object which contains the original drops. The original 2 drops are mixed into the new drop and the original drops can no longer be recovered. Alternatives, for example, letting drop2 merge into drop1 or the other way around, look arbitrary. Does that mean that any coalescing drops should always form a new drop even when one drop is significantly larger than the other? If this were not the case then an arbitrary, relative size would be needed to establish when the larger drop is so much larger that it absorbs the smaller one.

In the golf script club and ball both start deforming and energy starts to be transferred at the moment contact is made. It seems reasonable to treat the club and ball as a single new object in the short period they have contact and energy transfer is taking place. Alternatives are to let the club include the ball or the ball the club. Again choosing one or the other looks arbitrary and does not improve or simplify the script.

In his book [2] *Waves and Stones: On the Ultimate Nature of Reality* Graham Harman describes how a stone is thrown into an approaching wave. The Pool and Stone scenario is similar.

Here is the simplified scenario:

1. A stone falls through air towards a pool of water.
2. It touches the water and moves into it until it is fully immersed.
3. It falls to the bottom of the pool.
4. Water and air motion caused by the passage of the stone damp out.

By analogy with the drops and golf scenarios the water and air objects are defined as the volumes in motion because of the passage of the stone. Here is the above PoolAndStone scenario including these objects:

1. Before the stone touches the water, the air and stone are a Compound object called `stone_air` and the water object does not exist.
2. When the stone touches the water, the air SO, water SO and stone form a new Compound object called `stone_air_water`.
3. The water SO has no volume when it is created and grows rapidly as the stone moves into and through the water.

Before contact with the stone, the pool is a continuum. On contact with the stone the water object becomes a discrete, increasing, volume as the stone enters and moves through the water.

4. When the stone is just fully immersed, the air object separates from the Compound and becomes an independent object.

Objects water and stone continue as a new Compound called `stone_water`.

5. The air object begins to shrink as viscosity damps the disturbances caused by the passage of the stone. (Interaction between air and water at the water surface and interaction between the air object and the surrounding air are ignored).

6. Eventually the air is at rest and the air SO has zero volume and ends.

7. When the stone comes to rest at the bottom of the pool, the stone_water Compound object ends and the water and stone SOs are separate.

The stone comes to rest and the water and stone compound object splits into separate water and stone objects because there is no longer interaction between them, or to put it another way, the stone no longer transfers energy to the water.

8. The water object begins to shrink as viscosity damps the motion caused by the passage of the stone. (Interaction between the water object and the surrounding water has been ignored).

9. Eventually the water is at rest and the water object ends: All the water in the pool is again part of the continuum.

All Compound objects are 'new' objects in the PoolAndStone scenario. For example, stone_air is a new object which includes stone and air and stone_air_water is an object which includes stone, air and water, see the following figure.

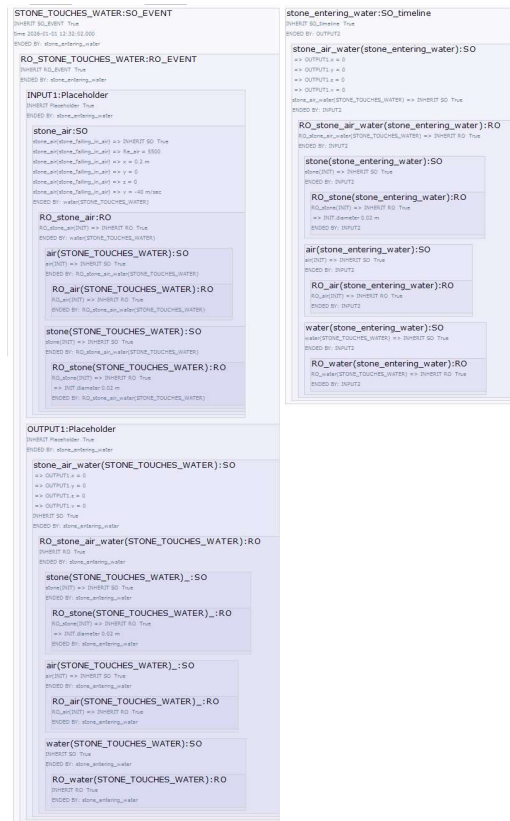


Figure 10: The Stone touches the Water and the stone_air_water Compound object is created.

The PoolAndStoneObject.txt script is an alternative PoolAndStone scenario and tests the idea of defining a dominant object and merging other objects into it. Instead of making Compound objects, stone_air, stone_air_water, stone_water and stone, the stone is given a 'dominant role'. The stone retains its identity as a stone and air and water are included and excluded as the script progresses. This approach simplifies and feels natural because the stone includes entrained air and water when they are still in contact with the stone. However if the air – water interaction at the surface of the pool were included in the script complications would arise if these damped out after the stone came to rest at the bottom of the pool. In a shallow pool it is likely that energy transfer between water and air, as ripples spread across the pool, is significant after the stone has come to rest. Similar complications occur when trying to make object air or water play a 'leading role'.

Another approach is to include stone_air as a Compound object in the object stone_air_water. This would lead to problems if properties depend only on air and water or stone and water.

This discussion implies that Compound objects should be created by combining other objects to form a new object without any internal hierarchy. This approach does not involve decisions on which object is dominant or problems when one object leaves a compound object which may itself be part of another compound object. For example 'air' has to leave stone_air_water which is a compound of stone_air and water. However this decision comes with a fairly serious objection; a collision between a large drop and an extremely small one must result in a new object and likewise the result of a rain drop falling into a large pool of water must also be a new object.

OOO seems to suggest that an object could include another object and retain its identity if its essential nature was retained. When considering coalescence of water drops this seems to imply that there is an arbitrary boundary, on the relative size of drops. A radical approach is to allow Compound objects with no internal hierarchy to be surrounded by sensual objects with all combinations of the contained real objects. A common practice in OOP is to create 'container' classes around other domain classes, defining interfaces which have to be implemented. This can often lead to information hiding to improve reliability. While not supported in the current version of WOOOF these techniques are appropriate in ensuring that OOO withdrawal is correctly modelled.

3.10 Bringing it together: drops scenario as an Example

The above points are illustrated using the drops scenario, see Figure 1, which shows two drops colliding and then splitting. The following figure shows the WOOOF presentation of the collision.

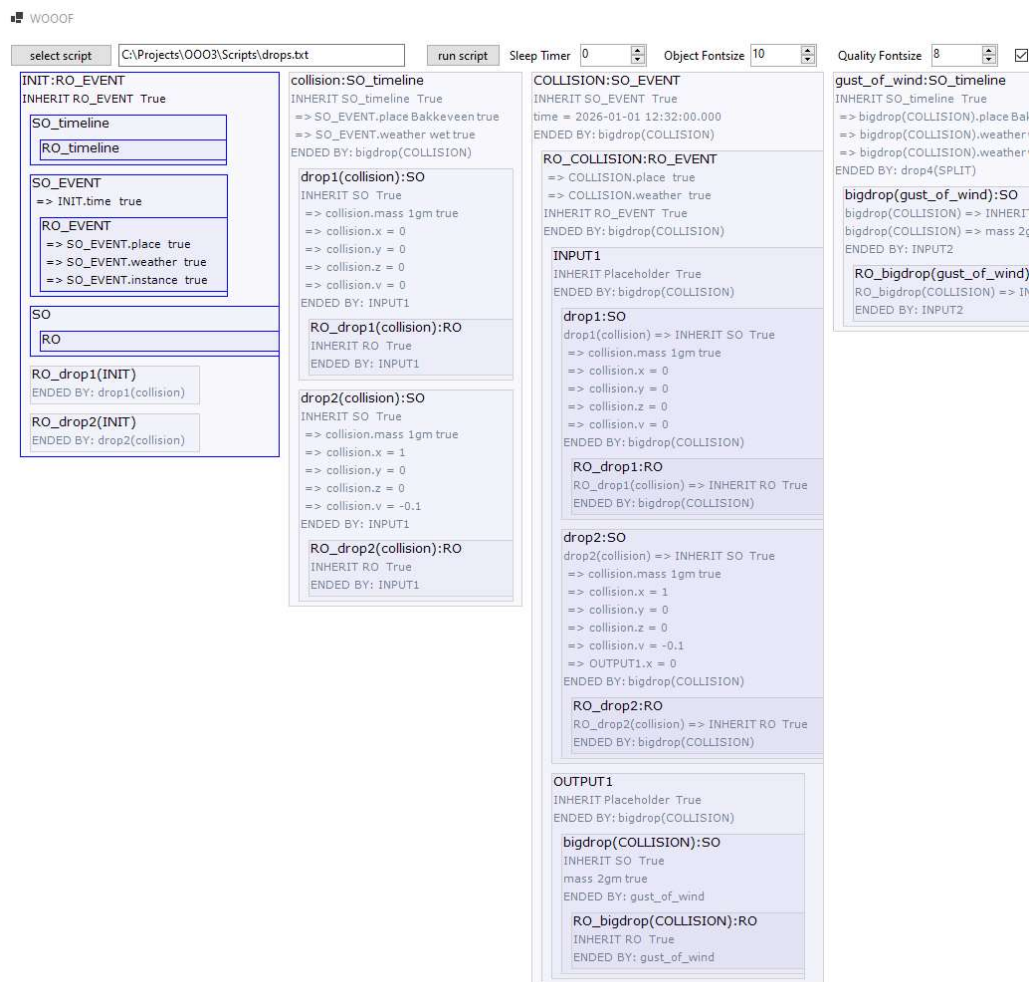


Figure 11: WOOOF output of drops.txt

The first block shows the INIT Event which introduces several object the most important being the two drops. The 2nd block shows the 'collision' Timeline which captures the movement of drop1 and drop2 on a collision course. These drops include in respectively RO_drop1 and RO_drop2.

The 3rd block shows the COLLISION. RO_COLLISION is included in the SO COLLISION and the vicarious causation is shown included in the RO_COLLISION. The INPUT of the Event is drop1 and drop2 and the OUTPUT is bigdrop, an SO which includes the newly created bigdrop and RO_bigdrop.

3.11 Instantiation

To close this section a brief discussion of instantiation. Instantiation of a class in OOP can be used for processing data. Consider a class describing an invoice (unique invoice number, date, sender, receiver, a list of products being billed, ...). As each new invoice is created an object is instantiated from the class using parameters such as invoice number, date, sender etc. The resulting invoices would all have the structure defined in the class.

A similar approach works in OOO. Imagine many pairs of drops colliding and coalescing in a thunder storm and a simplified version of the drops script with only coalescence and no drop splitting. This simplified 'drops' script could be converted into a class with parameters drop sizes, collision time, coordinates, velocities, etc., and could be instantiated for each of the pairs of colliding drops. It is also possible to imagine the output of two of these scripts being used as the input for another, creating a chain reaction of small drops coalescing to form 'rain' drops.

All objects in the scenarios are static classes meaning they cannot be instantiated or to put it another way they are both a class and a single object.

4 A Scenario with Observers.

A second type of scenario introduces two observers, a human and a dog. They both perceive and influence each other. Parallel sub-scenarios are used to represent a delayed response of the human to an action taken by the dog and vice versa: The actions of the dog occur in one sub-scenario and the delayed response of the human in another sub-scenario.

A simple model of information storage in the observers has been introduced. It is also apparent that the communication of observers is inherently more complicated than communication between inanimate objects: Inanimate Events always involve object collisions, object splitting or object property changes. Events in observers involve other types of interaction such as speech, responses to sight, smell, sound, feel and interactions motivated by memory or imagination.

4.1 The dogs Scenario.

The dog sits then runs and later barks at the same time as running. The human observer only sees the dog when it has started running and then later hears it barking. The human observer is a separate 'placeholder' with Timelines and Events which mirror the dog Timelines and Events but with a delay to represent the human observer seeing these Events shortly after they have occurred. The human observer, the 'placeholder', is called 'Me' in the scenario and what follows.

'Me', is used to simulate the delay between something happening and the observer seeing it. This delay is modelled in the human observer with the following Events and Timelines, see next figure:

Event: SEE_SOMETHING black moving.

Timeline: something_running

Event: SEE_RUNNING_DOG: Identify it as a running dog.

Timeline: dog_running

Event:SEE_NIDA_RUN: It is the dog called Nida. Change the property of 'Today_Nida' to running.

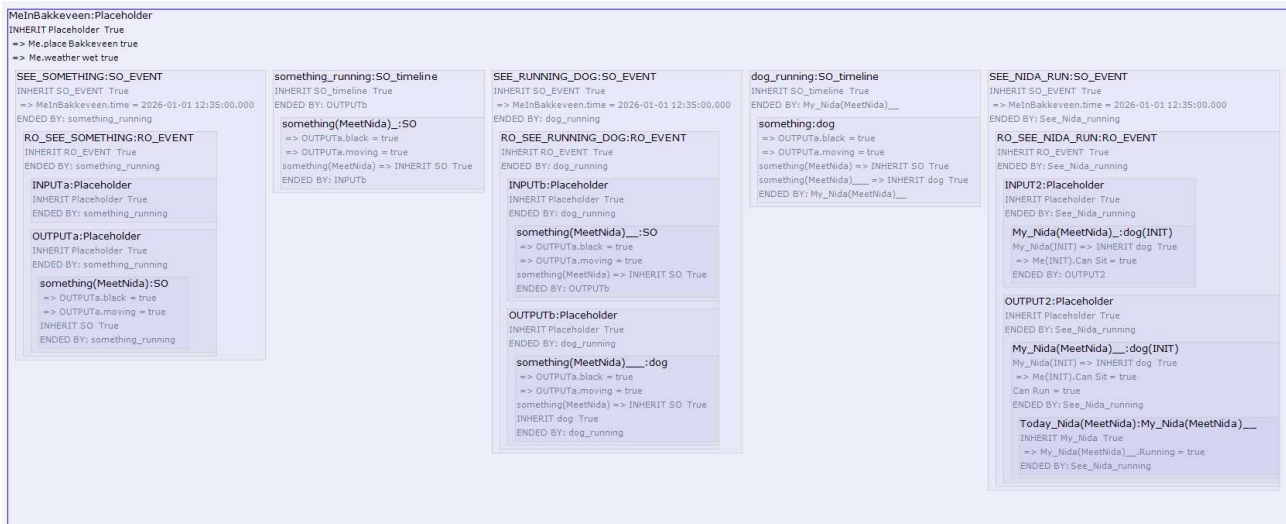


Figure 12: SEE_SOMETHING black moving.

Later the 'Me' object issues a command to sit. The dog interprets the command using a similar series of events and eventually sits.

The script uses a My_Nida object to captures everything 'Me' already knows about Nida. When 'Me' identifies the dog Nida he creates a new object called Today_Nida which tracks his experiences of Nida on the visit to Bakkeveen (a place in Friesland). Today_Nida is included in My_Nida as well as being a type of My_Nida, see the following figure.



Figure 13: My_Nida contains Today_Nida and Today_Nida is a type of My_Nida.

Inclusion of Today_Nida in My_Nida feels correct because the experience of My_Nida on that day are a part of My_Nida but are also separate as they relate to the events of that day. On completion of this part of the scenario My_Nida includes an encapsulated memory of the days events stored in Today_Nida.

If 'Me' sees Nida do something for the first time, for example jump 2 meters into the air, the Today_Nida would record this using an Event and would then update My_Nida, in another Event, with a new property: "can jump 2 meters into the air".

It looks promising to introduce an observer into the PoolAndStone script. We could imagine an observer anticipating a stone touching the surface of a pool of water and the observer claiming that the event occurred before it did, instead of afterwards.

4.2 Object hierarchy and Inheritance inside an Observer.

Inheritance is used to organise observer information, for example, the human has an object 'animal' which is parent to the objects 'human' and 'dog' and 'dog' is parent to a dog called My_Nida. The object 'dog' has properties which apply to all dogs. This is illustrated in the following figure which shows the object hierarchy as shown by WOOF.



Figure 14: My_Nida is a dog. dog is an animal.

An observer has information from direct experience (subjective experience) and information acquired from sources (objective insights) such as books, influencers and teachers. For example I experience a yellow daffodil subjectively and I also know that what I call yellow is my way of experiencing a narrow band of wavelengths of the electromagnetic spectrum. I am inclined to apply objective insights to interactions. In my case the source would be science while others may rely on magic or the influence of a god.

In the dogs script the object 'dog' has properties representing information which applies to all dogs. Storing information about animals which applies to dogs and other animals and information about dogs which applies to any dog, including one called Nida, is an efficient way to store information and makes it available for rapid interpretation of new experiences. I have discussed this with a friend who is an orthopedist. He sees that some patients with an autistic condition are not always able to create these generic objects resulting in

confusion if a new dog appears or that they are sometimes unable to use information about, for example, a red bus when they see a yellow one.

In OOP a parent object may be an abstract object implying that it cannot be implemented. What this could mean in a scenario is illustrated by an example: If the parent is 'dog' then a child may be an actual dog called My_Nida. The 'dog' would be abstract if we accept that there can be no actual 'dog' walking around.

In general one object including another object using composition implies a loose coupling between the objects. Inheritance implies a tight coupling and a tight coupling implies more changes to software when compared with loose coupling, if changes in requirements occur.

WOOOF is useful for testing alternative approaches. In some earlier versions of the dogs scenario, information in an observer was represented using composition instead of inheritance. Inheritance results in a more natural implementation and can also be used to show that the observer, called 'Me', realises he is a type of 'human'.

4.3 Visibility of Properties.

In OOP languages such as C#, properties of a parent class may be Private or Public. Private properties are not accessible from the child class but Public properties are.

At this time there is no explicit Private/Public modifier in WOOOF but properties can be overridden. The parent 'classes' such as animal, dog and human have properties which apply to most of the derived objects, for example, a 'dog' represents what I may know about dogs and could have properties:

```
dog has the property    BackLeftLeg
dog has the property    FrontRightLeg
dog has the property    BackRightLeg
dog has the property    Barks [= true    //true is assumed if there is no true or false specified.]
```

If My_Nida does not bark the bark property of dog is overridden

```
My_Nida has property Bark = false
```

If this override is not present then My_Nida inherits the Barks property from 'dog'; she barks!

4.4 Observers and OOO

I have not been able to find an explicit reference to application of OOO to observer experience or a comparison of objects in- and outside humans in [1] and [2]. However my interpretation of the first part of chapter 3 on the American Civil War in [1] is that

The American Civil War is a RO.

Graham Harman has an extensive and detailed SO of that War.

He has written a RO which is the first part of chapter 3 of [1] on that War.

I had an SO of the War before reading [1], having read on the subject.

I augmented my American Civil War SO while reading the first part of chapter 3 of [1].

I used that knowledge and internet to create the RO scenario called ACW.txt.

My conclusion is that OOO accepts that humans have SOs but does not attempt a comparison of SOs in and outside humans. This implies that abstract parent objects like 'animal' and 'dog' and interface objects which process information play a role which is not explicit in OOO. Attempting this step offers ways to analyse information storage and retrieval in animals using relatively concrete objects and OOO ideas.

5 Do the scenarios support OOO?

5.1 Level of Detail and Withdrawal.

The discussion on level of detail suggests a practical approach. When writing a scenario for Brownian motion there seems little point in creating Events for the arrival of every photon on the surface of a pollen particle.

The same conclusion is reached by comparing the rate of energy transfer caused by a photon and the collision of a molecule of water with the pollen. Practical limitations when writing scenarios or limitations introduced because of the intention of analysis do not necessarily influence the level of detail that reality might apply if real objects do exist.

However these considerations support the OOO idea of withdrawal and that it applies to all objects including the time objects introduced in these scenarios. In a certain sense we could turn the question around, realising that OOO encourages practical scenarios because of its acceptance of withdrawal.

5.2 Timelines and Events. Two types of object?

The Timelines have been implemented as SOs and as chunks of time derived (inherited) from the single Real continuous Timeline. This looks logical particularly when Timeline SOs overlap when parallel independent sub-scenarios are part of a scenario.

Events are instantaneous and can as a consequence involve no more than two objects; a way of avoiding complication. However it does look as though this introduces two types of time objects in addition to the single object type proposed by OOO.

5.3 A Generic Approach to Inanimate Interactions.

Dividing time into Timeline and Event SOs and the use of less obvious types of SO in the PoolAndStone script both contribute to a uniform approach to inanimate interactions. A uniform approach supports OOO and could even be considered a requirement for OOO.

In OOP the idea of a Pattern is used to capture the uniformity of similar processes. I think that at an explicit 'OOO' Pattern could be developed and tested against many inanimate scenarios lending further support to OOO.

5.4 Compound Objects.

When two or more objects combine a new real object should be created without an internal hierarchy. This approach is close to the OOO idea of a flat ontology.

However OOO seems to suggest that an object could include another object and retain its identity if its essential nature was retained. When considering, for example, coalescence of water drops this seems to imply that there is, what looks like an arbitrary boundary, on the relative size of drops. At some point the resulting drop is not a new one but the large existing one with a small addition.

Maybe withdrawal is telling us that we do not know how a compound real object is structured. Allowing SOs of all combinations of real objects in a compound object is a way of avoiding having to use arbitrary boundaries in sensual objects.

6 If OOO does describe reality or, at the very least, is a useful analysis tool; what questions arise from its success?

6.1 Why do Objects Work Outside and Inside an Observer?

Objects outside and inside an observer are both useful for analysing interactions and that raises the question why objects with similar characteristics work in both cases? If objects work in an observer while we know that neural networks are the underlying architecture then we could expect a mapping between the networks and objects. Trying to define this mapping looks useful because objects offer an accessible interface for humans while neural networks are often difficult to analyse.

The mirror effect of objects in several interacting observers and the mirroring of real external objects in objects in an observer lends some credence to the idea that the evolution of observer cognisance has been driven by real objects.

In the OOP C# language an "abstract" object, meaning an object which cannot be instantiated, is often used to define functions, properties and interfaces all child objects have (to have). In OOO a real abstract object which is inherited by all real objects will not exist although it may be possible to imagine one. However the OOO idea that real objects interact using different, context sensitive sensual objects is a useful tool for analysis and supports the idea of hidden information in real objects in a natural way.

6.2 An 'interface' Object which Observers use when Interpreting Reality

An attempt to model object processing using an 'interface' object which observers use when interpreting reality, was introduced in the dogs script; for the human and the dog. This type of object could be responsible for 'priming' existing SOs when a new experience occurs. Another comparable object could also be responsible for updating existing SOs if a new property occurs during an experience, see above: Nida can "jump 2 meters into the air". It looks improbable that this could work in inanimate objects like a water drop. This seems to suggest that a flat ontology is less likely to apply when including objects within observers.

7 Conclusion

A limited number of OOO characteristics have been addressed using WOOOF to write and display scenarios representing interactions between objects.

The three inanimate scenarios illustrate drop coalescence, a golf club hitting a ball and a stone hitting the surface of a pool and sinking to the bottom of the pool. They have a uniform approach to objects and two time objects, called Timeline and Event, play a leading role with no explicit parallel in OOO. The OOO idea of vicarious causation uses the instantaneous Event.

The Event simplifies scenarios and application to scenarios with many Events such as Brownian motion or atomisation instead of coalescence of water drops, should work because the instantaneous nature of Events ensure that a maximum of two objects are modified in each Event.

Finding a generic approach to inanimate interactions looks promising and may lead to a 'pattern' which can be applied to any inanimate scenario.

The attempts to standardize and simplify Compound object formation and disintegration, ran into a problem with avoidance of an arbitrary ratio of relative size. When should the object formed by a collision be a new object and when should it be a modified version of one of the original objects? Allowing SOs of all combinations of real objects in a compound object is way of avoiding this type of arbitrary boundary.

Finding a practical level of detail in all types of object (inanimate, timeline, Event) is a decision which is always relevant when developing a scenario. The OOO idea of withdrawal of real objects is one guide to finding a practical level of detail in scenarios.

The scenarios include a Real Object in its Sensual Object making use of the OOP concept of information hiding to implement withdrawal. Testing with a scenario where a Real Object requires more than one Sensual Object looks like a logical next step.

A speculative application of OOO to the processing of objects within an observer leads to several interesting complications. WOOOF lends itself to showing parallel timelines to represent delays between observers.

Comparing the observer scenario with the inanimate scenarios emphasises that the latter Events always involve physical interaction, such as collision, while observers have a wider range of communication.

Information in an observer uses inheritance and adds a new object type containing properties which, for example, apply to all dogs or an object animal which applies to all dogs and humans.

The observer scenario includes two examples of information processing using objects. These are another example of objects which differ from objects rooted in reality and at odds with the OOO idea of a flat ontology.